macOS Management Runbooks & IT-Support Enablement (Intune)

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary 4

The problem . . . 4

The solution . . . 4

What's included . . . 4

Why it matters . . . 4

Bottom line . . . 5

Solution Overview 5

What this solution is . . . 5

The macOS support operating model . . . 5

How it maps to an Intune-managed Mac fleet . . . 6

How the runbooks are structured . . . 6

Keeping the library current with Intune and Apple changes . . . 7

Requirements 7

R1. Intune role with read access to Mac devices (technician role) . . . 8

R2. Graph read access for the reporting scripts . . . 8

R3. A remote-control tool for Macs (with user consent) . . . 9

R4. Asset and user context (who owns which Mac) . . . 9

R5. A place to record runbook outcomes (ticketing / notes) . . . 9

Runbook Library 10

Runbook 1 — ADE / enrollment . . . 10

Runbook 2 — SSO setup (Platform SSO / Enterprise SSO) . . . 12

Runbook 3 — App deployment (VPP / line-of-business) . . . 13

Runbook 4 — Passwordless / FileVault procedures . . . 15

Runbook 5 — BYOD (user-driven / Company Portal enrollment) . . . 16

Runbook 6 — Compliance & Conditional Access . . . 17

Runbook 7 — Incident response (encryption / SecureToken / lock-out) . . . 18

Cross-runbook notes . . . 20

Testing 20

How to test a runbook . . . 20

Pass/fail gates per runbook . . . 21

Script tests (read-only, run first) . . . 22

Troubleshooting 23

Symptoms that cross runbook boundaries . . . 23

Confusing single-symptom cases . . . 23

When scope changes the answer . . . 24

Tooling and script edge cases . . . 24
When nothing fits . . . 25
Best Practices 25
Documentation . . . 25
Training and enablement . . . 25
Feature evaluation (adopting new capabilities deliberately) . . . 26
Escalation . . . 26
Live-environment safety (Mac specifics) . . . 27
Keeping the library healthy . . . 27
Deliverables Summary 27
Documents . . . 27
PowerShell scripts . . . 28
How the pieces fit together . . . 28
Outcome . . . 29
Appendix A — PowerShell Scripts 30

Executive Summary

The problem

Managing and supporting Macs gets **inconsistent** when there are no standard runbooks and the knowledge lives in one engineer's head. Two technicians handed the same Mac ticket enroll it, configure SSO, or recover it differently, escalate at different points, and capture different (or no) evidence. New Intune and Apple features get adopted ad hoc, if at all. The result is inconsistent support, knowledge silos, and — on a platform where **FileVault means a wrong move can lose a user's data** — real risk.

The solution

The **macOS Management Runbooks & IT-Support Enablement kit for Intune** gives every technician a repeatable, click-to-implement path for the common macOS management and support categories. It is built for an Intune + Entra + Apple Business Manager environment and is deliberately white-label and vendor-neutral: the Microsoft and Apple steps are documented at exact detail, and third-party product internals are deferred to those vendors' own documentation.

Its foundation is one simple operating model applied everywhere: **intake** → **triage** → **diagnose** → **resolve/escalate** → **document.** Each runbook has a fixed anatomy — a **triage tree**, ordered **checks**, a **resolution / setup steps**, an explicit **escalation gate**, and the **evidence to collect** — so any technician reaches the same correct outcome.

What's included

- **Seven categorized runbooks:** ADE/enrollment, SSO setup (Enterprise SSO / Platform SSO), app deployment (VPP/LOB), passwordless & FileVault procedures, BYOD, compliance & Conditional Access, and incident response (encryption / SecureToken / lock-out recovery).
- **Five PowerShell scripts:** Mac fleet health/context discovery, per-device state validation, a support-metrics/health report (all read-only), plus safe common-fix remediation (Sync / Rotate FileVault key) and a target-group builder (both default to `-WhatIf`).
- **Requirements, Testing, Troubleshooting, and Best Practices** guides — with pass/fail gates per runbook and guidance for documentation, staff training, and deliberate feature evaluation.

Why it matters

- **Consistency:** every technician manages and supports a Mac the same way.
- **Enablement:** the knowledge moves off one engineer's head into a trainable library, and new Intune/Apple features get adopted deliberately, not accidentally.
- **Safety in a live environment:** least-disruptive fixes first; **FileVault recovery key confirmed before any encryption-adjacent action**; destructive actions require approval and backup; state-changing scripts default to `-WhatIf`.
- **Reuse and cost:** a library and scripts you keep and run across the fleet — a fraction of the

cost of ad-hoc consulting.

Bottom line

Give every technician a repeatable way to **enroll it, configure it, support it, and recover it** — and to adopt new features deliberately — across the Mac fleet, end to end.

Solution Overview

What this solution is

This is a **runbook library and enablement kit** — a set of standardized, step-by-step procedures for managing and supporting **Intune-managed Macs**, plus the training and feature-evaluation guidance that keeps a support team consistent over time. Instead of every technician improvising (or the knowledge living in one engineer's head), each common macOS task and problem category gets a single repeatable path: **triage tree** → **checks** → **resolution** → **escalation gate** → **evidence to collect**.

The goal is simple: any technician, at any skill level, can pick up a Mac ticket or a Mac onboarding task, follow the matching runbook, and either complete it the same way every other technician would, or escalate it with the right evidence already attached. macOS work gets **owned end to end** instead of bounced or improvised.

This library is deliberately **white-label and vendor-neutral**. It documents the Microsoft Intune, Microsoft Entra, and Apple Business Manager (ABM) steps at exact click-by-click detail. Where a third-party product (a specific SSO extension, a specific line-of-business app, an MDM migration tool) is involved, the runbook tells you *what* to check and *when* to hand off to that vendor's own documentation, rather than guessing at another product's internals.

> ***A note on macOS specifics:*** *macOS management has moving parts Windows does not — Automated Device Enrollment (ADE) driven by Apple Business Manager, FileVault and its* ***SecureToken / bootstrap token*** *model, Platform SSO, VPP app licensing, and local-account nuances. Where a step depends on a fast-changing Apple or Intune behavior, the runbook flags it and points you to the vendor documentation to confirm before relying on it.* ***Flagged as uncertain:*** *exact menu paths in Apple Business Manager and the Intune admin center change periodically — confirm the current path in the console if a label has moved.*

The macOS support operating model

Every runbook in this library follows the same five-stage operating model. Learn it once and it applies to every category.

1. **Intake** — Capture the ticket or task cleanly. Who is the user, which Mac (Intune device

name / serial), what exactly is happening or is being requested, when did it start, and how many people are affected (one user, one device, one location, or the whole Mac fleet). Scope decides everything downstream.

1. **Triage** — Use the runbook's **triage tree** to classify the issue into a category and a

likely cause branch. Triage answers: *which runbook applies, and which branch of it?*

1. **Diagnose** — Run the runbook's **checks** in order. Each check is a read-only observation

(a signal in Intune, Entra, ABM, or on the Mac) that confirms or rules out a cause. You stop when a check identifies the cause.

1. **Resolve or Escalate** — If the identified cause has a **resolution** in the runbook, apply

it (least-disruptive fix first). If the cause is past the runbook's **escalation gate** — it needs rights you don't have, changes that affect others, or another team's system (and, on a Mac, anything that risks FileVault data) — escalate.

1. **Document** — Record what you observed, what you did, and the outcome, using the runbook's

evidence to collect list. Good documentation is what lets the next technician (or the escalation target) continue without redoing your work — and it feeds the enablement loop (see Best Practices).

```
INTAKE  ->  TRIAGE  ->  DIAGNOSE  ->  RESOLVE ■■ or ■■> ESCALATE
  |          (tree)     (checks)    (fix)              (gate)
  +----------------------- DOCUMENT (evidence) -------------------+
```

How it maps to an Intune-managed Mac fleet

On an Intune-managed Mac fleet, the technician has four sources of truth, and every runbook draws on them in the same order:

• **Microsoft Intune admin center** (`https://intune.microsoft.com`) — the Mac's management

state: enrollment, compliance, assigned configuration profiles and apps, hardware inventory, and remote actions such as **Sync**, **Restart**, **Rotate FileVault key**, and **Collect diagnostics**.

• **Microsoft Entra admin center** (`https://entra.microsoft.com`) — the identity side: the user

account, group memberships (which drive policy/app targeting), sign-in logs, Conditional Access, and device registration state.

• **Apple Business Manager** (`https://business.apple.com`) — the Apple side of enrollment: which

Macs are assigned to the Intune MDM server, VPP app licenses (**Apps and Books**), and the ADE token. This is unique to Apple platforms and has no Windows equivalent.

• **The Mac itself** — reached through an approved remote-control tool (with user consent) or

read-only signals; local tools such as **System Settings** → **General** → **Device Management**, the **Company Portal** app, **Console.app**, and Terminal commands like `profiles status`, `fdesetup status`, and `dsmemberutil`.

Because targeting on this fleet is **group-based**, a recurring diagnostic question across almost every runbook is: *is this Mac/user in the right Entra group to receive the profile or app in question?* The runbooks make that check explicit rather than assuming it.

How the runbooks are structured

Every runbook in the Runbook Library uses the same fixed anatomy so a technician always knows where to look:

• **Scope / symptoms** — the user-visible symptoms or the task this runbook covers, and what it

does *not* cover (with a pointer to the correct runbook).

• **Triage tree** — a short branching decision that narrows the issue to a probable cause path and tells you which checks to run first.

• **Checks** — an ordered list of read-only observations. Each check names exactly where to click and what a healthy vs. unhealthy result looks like.

• **Resolution** — the fix (or the setup steps) for each identified cause/branch, least-disruptive first, written as click-to-implement steps.

• **Escalation gate** — the explicit line: what conditions mean *stop and escalate*, to whom, and what to hand over. On a Mac this includes anything that risks FileVault data loss.

• **Evidence to collect** — the specific artifacts (screenshots, IDs, log names, command output, timestamps) to attach to the ticket regardless of outcome.

Keeping the library current with Intune and Apple changes

macOS management changes faster than Windows: Apple ships a major macOS release yearly (and drops support for MDM commands and hardware on its own cadence), and Intune's macOS feature set is updated continuously. This library is built to be **maintained**, not frozen:

• **Watch the release notes.** Track the Intune "What's new" and Apple's macOS/ABM release notes. When a menu moves, a command is deprecated, or a new capability lands (e.g. a new Platform SSO mode), update the affected runbook.

• **Re-verify on each major macOS release.** Before the fleet adopts a new macOS version, re-run the Testing gates for enrollment, FileVault, SSO, and compliance on a pilot Mac.

• **Feed corrections back.** When a technician finds a step that no longer matches the console, the correction goes into the runbook so the next technician benefits (see Best Practices — Feature evaluation and Keeping the library healthy).

The remaining documents build on this overview: **Requirements** (what a technician needs before using the library), the **Runbook Library** itself (the categorized runbooks), **Testing** (pass/fail gates that prove each runbook works), **Troubleshooting** (cross-cutting edge cases), and **Best Practices** (documentation, training, and feature evaluation).

Requirements

Before a technician can work the runbooks in this library, a small set of prerequisites must be in place. These are **in addition to** the category-level prerequisites in `global-requirements-infrastructure.md` (Entra tenant, Intune subscription, device enrollment, network access to Microsoft cloud endpoints), which you should confirm first.

> ***macOS note on the global requirements:*** *the category-level global requirements are written for a Windows baseline (they mention Windows 10/11 and Windows auto-enrollment). For this solution, read those as their macOS equivalents: a* ***supported macOS release*** *(a current or prior-year macOS*

*version that Apple and Intune still support) in place of the Windows baseline, and **Automated Device Enrollment (ADE) via Apple Business Manager** (or user-driven Company Portal enrollment) in place of Windows automatic enrollment. The identity, subscription, role, pilot-group, and network requirements apply unchanged.*

Each requirement below is written as **Description / Why / Verify / Implement if missing** so you can confirm it click-by-click and fix it if it is not there.

R1. Intune role with read access to Mac devices (technician role)

Description: The technician's account holds an Intune role that can read device inventory, compliance, configuration, and app assignments — at minimum **Help Desk Operator**, or a custom role scoped to the same read actions plus the remote actions the team is allowed to use (Sync, Restart, Rotate FileVault key, Collect diagnostics).

Why: Every runbook's **checks** are read operations against Intune (device state, compliance, assigned profiles). Without a role that can read these, the technician is blind and every ticket becomes an escalation.

Verify: Sign in to `https://intune.microsoft.com`. Go to **Tenant administration** → **Roles** → **All roles**, open the role assigned to the technician, and confirm it includes read for **Managed devices**, **Device configurations**, and **Mobile apps**. Also open **Devices** → **All devices**, filter to **OS = macOS**, pick any Mac, and confirm you can open its **Overview**, **Hardware**, and **Device compliance** blades.

Implement if missing: In Intune → **Tenant administration** → **Roles** → **All roles**, select **Help Desk Operator** → **Assignments** → **Assign**. Give the assignment a name, add the technicians' Entra group under **Admin groups**, set **Scope (groups)** to the Mac device groups they support, and choose the **Scope tags** if used. Save. Have the technician sign out and back in.

R2. Graph read access for the reporting scripts

Description: An account (or app registration) that can sign in to Microsoft Graph with the **read** scopes the bundled PowerShell scripts request: `DeviceManagementManagedDevices.Read.All`, `DeviceManagementConfiguration.Read.All`, `DeviceManagementApps.Read.All`, and `Directory.Read.All`.

Why: The Discovery, Validation, and Report scripts pull Mac health and context from Graph so a technician can triage across many Macs at once instead of clicking each one. They are read-only and never write to the tenant.

Verify: On the technician's workstation, run `Connect-MgGraph -Scopes "DeviceManagementManagedDevices.Read.All"` and confirm sign-in succeeds and the account consents (or has admin-consented) the scope. Run `Get-MgContext` and confirm the scopes are present.

Implement if missing: Install the Microsoft Graph PowerShell SDK (`Install-Module Microsoft.Graph.Authentication -Scope CurrentUser`). If your tenant requires admin consent for these scopes, have an Intune Administrator or Global Administrator grant consent once (Entra → **Enterprise applications** → **Microsoft Graph Command Line Tools** → **Permissions**), or create a dedicated app registration with the read scopes above and admin-consent it. The Remediation and Set-ScopeTargeting scripts additionally require write scopes, listed in their own headers — grant those

only when you intend to make a change.

R3. A remote-control tool for Macs (with user consent)

Description: A tool that lets the technician view and, with user consent, control the Mac to run the on-device steps in the runbooks. Microsoft **Remote Help** support for macOS may not cover every scenario, so an organization-approved remote-control tool that supports macOS (and works with macOS Screen Recording / Accessibility privacy consent) is often used.

Why: Several checks and resolutions require driving the local Mac (opening System Settings, running `profiles status` or `fdesetup status`, reading Console logs). A sanctioned, audited remote tool is the safe way to do this.

Verify: On a test Mac, start a session with the approved tool and confirm the technician can view the screen and, with the user's consent, control it — including granting the macOS **Screen Recording** and **Accessibility** privacy prompts the tool needs.

Implement if missing: Deploy and configure the organization-approved macOS remote-control tool (and, where required, deploy its **Privacy Preferences Policy Control (PPPC)** profile via Intune → **Devices** → **macOS** → **Configuration** so the tool's screen/accessibility permissions are pre-approved). **Flagged as uncertain:** confirm current Remote Help macOS capabilities against Microsoft's documentation before assuming a scenario is covered.

R4. Asset and user context (who owns which Mac)

Description: A reliable way to map a caller to their Mac(s) and to the device's primary user, location, and hardware — the Intune **Primary user** field, device naming, the **serial number**, and any asset tag or CMDB the organization keeps. On Macs the **serial number** is especially important because it is the key that ties the device to Apple Business Manager and ADE.

Why: Intake and triage depend on identifying the exact Mac. Without primary-user mapping and the serial, a technician cannot confirm they are looking at the right machine, cannot check ABM assignment, and cannot establish scope (one user vs. many).

Verify: In Intune → **Devices** → **All devices** (filter **OS = macOS**), open a Mac and confirm the **Primary user** is populated, the **Device name** follows a recognizable convention, and the **Serial number** under **Hardware** matches the physical device (Apple menu → **About This Mac**, or on the login screen). Confirm you can search by serial.

Implement if missing: Ensure primary user is set on enrollment (it is by default for user-driven / user-affinity ADE). For shared Macs, document the exception. Adopt a device-naming convention and, if you keep a CMDB, record the Intune device name and serial there so callers can be matched to Macs. Confirm the serials of managed Macs also appear in Apple Business Manager (see Runbook 1).

R5. A place to record runbook outcomes (ticketing / notes)

Description: A ticketing system or shared notes location where the **evidence to collect** for each runbook is stored against the ticket, and where completed onboarding tasks are recorded.

Why: The operating model's final stage is **Document**. Consistent evidence capture is what makes escalations clean, lets trends surface in the Report script, and feeds the enablement / feature-evaluation loop.

Verify: Confirm the team's ticketing system has a field or attachment area for screenshots, device IDs/serials, command output, and timestamps, and that technicians know where the runbook evidence goes.

Implement if missing: Designate the evidence fields in your existing ticketing tool. If you do not have one, a shared, access-controlled location with a per-ticket note is the minimum. Standardize the fields to match the **evidence to collect** lists in the Runbook Library. **Never record FileVault recovery keys, passwords, or bootstrap-token material in a ticket** (see Best Practices — evidence capture).

Once R1–R5 and the category-level global requirements (read with the macOS note above) are confirmed, the technician is ready to work the Runbook Library in the next document.

Runbook Library

This is the heart of the solution: the categorized runbooks a technician follows to **manage and support Intune-managed Macs**. Each runbook uses the fixed anatomy from the Solution Overview — **Scope / symptoms**, **Triage tree**, **Checks**, **Resolution**, **Escalation gate**, **Evidence to collect** — so you always know where to look next. Some runbooks are *setup* tasks (enrollment, SSO, app deployment, BYOD) and some are *support/incident* tasks (compliance, incident response); the anatomy is the same for both — for a setup runbook the "resolution" is the step-by-step configuration.

How to use this library: From the ticket or task, pick the runbook whose *symptoms/scope* match. Work the *triage tree* to land on a branch, run the *checks* in order until one identifies the cause (or confirms readiness for a setup step), apply the matching *resolution/steps*, or stop at the *escalation gate*. Attach the *evidence* regardless of outcome. Always prefer the least-disruptive path first (Sync before Restart, targeted change before re-enroll, and **never** an action that risks FileVault data without approval and a confirmed recovery key).

Three conventions used throughout:

- **Intune** = Microsoft Intune admin center, `https://intune.microsoft.com`.
- **Entra** = Microsoft Entra admin center, `https://entra.microsoft.com`.
- **ABM** = Apple Business Manager, `https://business.apple.com`.

> ***Menu-path caveat (applies to every runbook):*** *Apple and Microsoft rename and move console menus periodically. Where a path is given, confirm the current label in the console if it has moved. macOS Terminal commands (*`profiles`*, *`fdesetup`*, *`dsmemberutil`*, *`sysadminctl`*) are run in* ***Terminal.app*** *and several require* `sudo`*; run them read-only first.*

Runbook 1 — ADE / enrollment

Scope / symptoms

Setting up or fixing Automated Device Enrollment (ADE) for Macs: a new Mac should enroll automatically through Setup Assistant but doesn't; a Mac is missing from Intune; enrollment fails or the enrollment profile isn't offered; a Mac enrolled but shows no management profile. **Not covered:** the Mac is enrolled but non-compliant (Runbook 6); user-driven Company Portal enrollment for BYOD (Runbook 5).

Triage tree

```
Is this a NEW device setup, or an EXISTING Mac that lost/failed enrollment?
■■ New setup   → is the serial assigned to the Intune MDM server in ABM?
■    ■■ No  → ABM assignment/token problem            → Checks C1–C2
■    ■■ Yes → enrollment profile / Setup Assistant     → Checks C3–C4
■■ Existing    → is the management profile present on the Mac?
     ■■ No  → profile removed / never installed         → Checks C4–C5
     ■■ Yes → visible in Intune but stale               → Check C5
```

Checks

• **C1 — ABM device assignment (ABM):** ABM → **Devices** → search the **serial** → confirm it is

assigned to the **MDM Server** that represents Intune. Unhealthy: serial not present, or assigned to a different/none MDM server.

• **C2 — ADE token / sync (Intune):** Intune → **Devices → macOS → macOS enrollment →

Enrollment program tokens → **the token** → **confirm it is** valid/not expired **and** Sync**; check the associated** enrollment profile**. Unhealthy: token expired, or no default profile assigned.

• **C3 — Enrollment profile assignment (Intune):** Intune → **Devices → macOS → macOS enrollment →

Enrollment program tokens → (token) → Profiles / Devices** — confirm an ADE enrollment profile is assigned to this serial. Unhealthy: device has no profile assigned.

• **C4 — Management profile on the Mac (device):** run `profiles status -type enrollment` and open

System Settings → **General** → **Device Management**. Unhealthy: "no configuration installed" / no MDM profile.

• **C5 — Presence & check-in (Intune):** Intune → **Devices** → **All devices** (OS = macOS) — is the

Mac there? Note **Last check-in**. Stale last check-in → not syncing (see Runbook 6 C-sync).

Resolution

• **Not in ABM / wrong MDM server (C1):** in ABM, assign the serial to the Intune MDM server

(**Devices** → **Edit MDM Server**), then in Intune **Sync** the token. If the Mac was bought outside Apple's reseller channel, it may need manual addition via **Apple Configurator** — hand to the team that owns ABM.

• **Token/profile (C2/C3):** if the token is expired, renew it (download a new token from ABM →

Preferences → **MDM Server** → **Download Token**, upload in Intune) — this is usually an admin task (see gate). Assign a **default enrollment profile** to the token so new serials receive it.

• **Setup Assistant didn't offer enrollment (C4):** the Mac must be **erased and reset** to re-run

Setup Assistant for ADE (data-loss — see gate) *or* enrolled user-driven via Company Portal as a fallback (Runbook 5). Do not erase without approval and backup.

- **Missing/stale profile (C4/C5):** for a Mac still in Intune, trigger **Sync** (Intune → device →

Sync) and on the Mac **System Settings** → **Device Management** to confirm the profile reinstalls; if the MDM profile was removed by the user, re-enroll via Company Portal.

Escalation gate

Escalate to **endpoint engineering / the ABM owner** when: the ADE token is expired and must be renewed; a serial isn't in ABM and needs Apple Configurator or reseller action; enrollment fails repeatedly with the same error; or the fix requires **erasing the Mac** (owner approval + confirmed data backup, and confirm you have the FileVault recovery key before any erase).

Evidence to collect

Serial number; ABM assignment (which MDM server) screenshot; token validity + assigned enrollment profile; `profiles status -type enrollment` output; **Device Management** screenshot; Intune Overview (Last check-in); exact enrollment error text.

Runbook 2 — SSO setup (Platform SSO / Enterprise SSO)

Scope / symptoms

Setting up or fixing single sign-on to Microsoft Entra on Macs: deploying the **Microsoft Enterprise SSO plug-in** (Company Portal-based) or **Platform SSO**; SSO to Microsoft 365 apps fails; repeated Entra sign-in/MFA prompts on a Mac; Platform SSO registration doesn't complete. **Not covered:** the account itself is disabled/locked in Entra (Runbook 7 identity paths); app install issues (Runbook 3).

Triage tree

```
Is this INITIAL setup or a BROKEN existing SSO?
■■ Setup   → which mode? Enterprise SSO plug-in (baseline) or Platform SSO (device-bound)
■            → follow the matching Resolution steps
■■ Broken  → is the SSO configuration profile installed and Company Portal present?
    ■■ No  → profile/app not delivered → Checks C1–C2
    ■■ Yes → registration/token issue  → Checks C3–C4
```

Checks

- **C1 — SSO profile assigned (Intune):** Intune → **Devices** → **macOS** → **Configuration** → the SSO

extension profile → **Properties** → **Assignments** — is the Mac's group targeted? Unhealthy: device not in the assigned group (fix via Set-ScopeTargeting).

- **C2 — Company Portal present (device):** confirm **Company Portal** is installed (the Enterprise

SSO plug-in ships with it) and the user has signed in once. Unhealthy: Company Portal missing or never signed in.

- **C3 — Profile installed on Mac (device): System Settings** → **General** → **Device Management** →

confirm the **Single Sign-On Extension** / **Platform SSO** payload is present; run `app-sso platform -s` (Platform SSO state) if using Platform SSO. Unhealthy: payload missing, or Platform SSO shows **not registered**.

- **C4 — Sign-in evidence (Entra):** Entra → **Users** → **(user)** → **Sign-in logs** — read the most

recent failure's **Failure reason** and **Conditional Access** tab. Unhealthy: token/MFA failing, or a CA policy blocking.

Resolution

- **Deploy Enterprise SSO plug-in (baseline):** Intune → **Devices → macOS → Configuration →

Create → Templates → Device features **(or** Settings catalog → Single Sign-On Extension**), add the** Microsoft Enterprise SSO plug-in **(extension identifier for the Microsoft SSO extension), set the required Entra values, assign to the Mac group. Ensure** Company Portal** is deployed (Runbook 3). Sign in to Company Portal once to activate it.

- **Deploy Platform SSO (device-bound):** create a **Platform SSO** configuration (Settings catalog)

choosing the authentication method (e.g. Secure Enclave key or password sync per your design), assign it, then have the user complete the **registration prompt** at login. **Flagged as uncertain:** Platform SSO options and prerequisites change across macOS releases — confirm the current supported settings before rollout, and pilot first.

- **Targeting/app missing (C1/C2):** add the Mac to the assigned group (Set-ScopeTargeting) or

deploy Company Portal; re-sync.

- **Registration/token (C3/C4):** have the user sign out/in or re-run the Platform SSO

registration; confirm device time is correct (token failures are often clock skew); if a CA policy blocks, that's an identity decision (gate).

Escalation gate

Escalate to the **identity/security team** when a Conditional Access policy must change, Platform SSO design decisions are needed (auth method, key type), or SSO fails tenant-wide after the profile and Company Portal are confirmed present.

Evidence to collect

SSO profile name + assignment; Company Portal present/signed-in yes/no; **Device Management** screenshot showing the SSO payload; `app-sso platform -s` output (if Platform SSO); sign-in log failure reason + CA result; device date/time.

Runbook 3 — App deployment (VPP / line-of-business)

Scope / symptoms

Deploying or fixing Mac apps via Intune: a **VPP (Apps and Books)** store app or a line-of-business `.pkg`/`.dmg` won't install, is missing, or fails; a required app (Company Portal, a browser, an LOB app) didn't reach the Mac. **Not covered:** the app installs but can't sign in (Runbook 2 / Runbook 7); the app can't reach its server (network — outside this library's scope, hand to the network team).

Triage tree

```
Is the app a VPP (store) app or a line-of-business package?
■■ VPP       → are licenses available and the token valid?
■   ■■ No  → VPP token / license problem → Checks C1–C2
■   ■■ Yes → assignment/targeting        → Checks C3–C4
■■ LOB pkg   → is it assigned and installing?
```

```
■■ No  → targeting/assignment        → Check C3
■■ Yes but failing → package/detection → Check C4
```

Checks

• **C1 — VPP token (Intune):** Intune → **Tenant administration → Connectors and tokens → Apple VPP tokens **— confirm the token is** valid/not expired **and** synced**. Unhealthy: expired or errored token.

• **C2 — License availability (Intune/ABM):** the VPP app → confirm **available licenses > 0** (ABM → **Apps and Books** shows purchased quantity). Unhealthy: licenses exhausted.

• **C3 — Assignment/targeting (Intune/Entra):** Intune → **Apps → macOS → (app) → Properties → Assignments **— is the Mac/user in the assigned group? Read** Device/User install status**. "Not applicable" almost always means the target isn't in the group.

• **C4 — Install status / detection (Intune/device):** the app → **Device install status** → read the state and error; on the Mac confirm whether the app is in **/Applications** and its version (matches the LOB detection rule?). Unhealthy: install failed, or detection rule doesn't match the delivered version.

Resolution

• **VPP token/licenses (C1/C2):** renew/re-sync the VPP token (admin task — see gate); buy more licenses in ABM → **Apps and Books** if exhausted; re-sync in Intune.

• **Deploy a VPP app:** Intune → **Apps** → **macOS** → **Add** → **macOS app (Apple VPP)** → pick the app → choose **device** or **user** licensing → assign to the Mac group. Sync.

• **Deploy an LOB app:** Intune → **Apps** → **macOS** → **Add** → **macOS app (PKG)** (or line-of-business format), upload the signed package, set the **detection/version** info, assign to the group. For unmanaged formats, prefer a signed `.pkg`. Sync.

• **Targeting miss (C3):** add the Mac to the assigned group (Set-ScopeTargeting) — do **not** sideload around the deployment.

• **Failed install / detection (C4):** trigger **Sync**; if it still fails, read the error; fix the detection rule to match the delivered version; for a product-internal install error, hand off to that app vendor's documentation with the evidence attached.

Escalation gate

Escalate to the **app owner / packaging team** when an app install fails fleet-wide, a package or detection rule is wrong, or a VPP token/licensing/entitlement issue needs an admin. Escalate to the **application vendor** for product-internal install faults beyond reinstall/repair.

Evidence to collect

App name + version; VPP token state and license count (if VPP); Intune install status + error code; whether the target is in the assigned group; whether the app is present in `/Applications` and its version; steps already tried.

Runbook 4 — Passwordless / FileVault procedures

Scope / symptoms

Configuring and supporting FileVault encryption and modern sign-in on Macs: enabling/enforcing FileVault via Intune, **escrowing and rotating the recovery key**, confirming **SecureToken** and the **bootstrap token**, and standing up passwordless sign-in (Platform SSO device-bound authentication). **Not covered:** a Mac is *locked out* or FileVault is *broken* (Runbook 7 — incident response); Platform SSO *setup* details (Runbook 2).

> ***Safety first (read before touching FileVault):*** *FileVault protects user data with encryption. Any action that could change the encryption state, the unlock account, or the recovery key can* ***lock a user out of their data permanently*** *if the recovery key isn't escrowed. Confirm the recovery key is escrowed in Intune* ***before*** *any FileVault change, and treat the key as a secret (never paste it into a ticket).*

Triage tree

```
Is FileVault being ENABLED/ENFORCED, or is this KEY/TOKEN maintenance?
■■ Enable/enforce → is a FileVault policy assigned and is the key escrowing?
■    ■■ No policy  → deploy FileVault policy → Resolution "Enable FileVault"
■    ■■ Policy on  → key not escrowed        → Checks C1–C2
■■ Key/token maint → rotate key OR verify SecureToken/bootstrap token → Checks C2–C4
```

Checks

- **C1 — FileVault policy assigned (Intune):** Intune → **Devices** → **macOS** → **Configuration** →

the FileVault (disk encryption) profile → **Assignments** — targeted at the Mac's group? Unhealthy: not assigned.

- **C2 — Recovery key escrowed (Intune):** Intune → **Devices** → **(the Mac)** → **Recovery keys** (or

Monitor → **Encryption report**) — is a **personal recovery key** stored? Unhealthy: no key escrowed (do not change FileVault until this is resolved).

- **C3 — FileVault state (device):** run `fdesetup status` — "FileVault is On/Off" and any

in-progress encryption. Unhealthy: mismatch with policy (policy says On, device Off).

- **C4 — SecureToken / bootstrap token (device):** `sysadminctl -secureTokenStatus <user>`

(SecureToken for the account) and `profiles status -type bootstraptoken` (bootstrap token escrowed to MDM). Unhealthy: user has **no SecureToken**, or **no bootstrap token** escrowed — future FileVault-enabled operations may fail.

Resolution

- **Enable FileVault (deploy):** Intune → **Devices → macOS → Configuration → Create → Templates →**

Endpoint protection → FileVault **(or** Settings catalog → FileVault**): turn on** Enable FileVault**, set** key type = Personal recovery key**, enable** Escrow recovery key**, choose the deferral behavior, and assign to the Mac group. FileVault engages at the user's next sign-in.

- **Escrow missing key (C2):** trigger **Sync**; if the key still doesn't escrow, use the Intune

device action **Rotate FileVault key** (this generates and escrows a new key) — confirm the new key appears under **Recovery keys**. If FileVault was enabled outside policy, the key may need to be rotated to

capture it.

- **Rotate the recovery key (maintenance / after exposure):** Intune → **Devices → (the Mac) →

Rotate FileVault key** (or the Remediation script), then confirm the new key escrows. Rotate any time a key may have been exposed.

- **SecureToken / bootstrap token (C4):** ensure the **bootstrap token** is escrowed (on ADE-enrolled

Apple silicon Macs it is captured automatically at first admin sign-in; confirm with `profiles status -type bootstraptoken`). If a user account lacks SecureToken, an admin account that has one can grant it with `sysadminctl -secureTokenOn <user> ...` — **flagged as uncertain and sensitive:** SecureToken grants are hardware/OS-version dependent and can affect who can unlock FileVault; do this deliberately on a pilot and escalate if unsure (see gate).

- **Passwordless (Platform SSO):** stand up device-bound Platform SSO per Runbook 2 so the user

authenticates with a Secure Enclave key rather than a password where your design supports it.

Escalation gate

Escalate to **endpoint engineering** when: the recovery key **cannot** be escrowed after rotation; `fdesetup`/SecureToken/bootstrap-token state can't be reconciled with policy; or any FileVault change is contemplated **without** a confirmed escrowed key (stop — data-loss risk). Never disable FileVault or reset an account's password in a way that could orphan the encryption without owner approval and a verified recovery key.

Evidence to collect

FileVault policy name + assignment; `fdesetup status` output; escrowed-key present yes/no (never the key value); `sysadminctl -secureTokenStatus` and `profiles status -type bootstraptoken` results; rotation confirmation (new key escrowed).

Runbook 5 — BYOD (user-driven / Company Portal enrollment)

Scope / symptoms

Onboarding a personally owned Mac (BYOD) or a Mac that must enroll user-driven rather than via ADE: enrolling through **Company Portal**, applying the lighter BYOD profile/app set, and confirming compliance without ADE. **Not covered:** corporate ADE devices (Runbook 1); the Mac won't sign in to SSO afterward (Runbook 2).

Triage tree

```
Is this a NEW BYOD enrollment or a BYOD Mac with a PROBLEM?
■■ New   → is Company Portal installed and enrollment allowed for this user?
■    ■■ No  → enrollment restriction / app missing → Checks C1–C2
■    ■■ Yes → walk the user through enrollment      → Resolution "Enroll via Company Portal"
■■ Problem → enrolled but not compliant / not getting apps → Checks C2–C3 (then Runbook 6)
```

Checks

- **C1 — Enrollment restrictions (Intune):** Intune → **Devices → Enrollment → Enrollment device

platform restrictions → macOS **— is personally owned macOS enrollment** allowed**? Unhealthy: personal macOS blocked, or the user isn't licensed for Intune.

• **C2 — Company Portal (device):** confirm the user downloaded **Company Portal** (Mac) from the documented source and can sign in. Unhealthy: app missing or sign-in fails.

• **C3 — Assignments for BYOD (Intune/Entra):** confirm the BYOD Entra group (and its assigned profiles/apps) targets this user/device. Unhealthy: user not in the BYOD group.

Resolution

• **Allow BYOD (C1):** Intune → **Devices → Enrollment → Enrollment device platform restrictions → macOS → **set** Personally owned **to** Allow** for the intended users; confirm Intune licensing.

• **Enroll via Company Portal:** have the user install **Company Portal** for Mac, sign in with their work account, and follow the prompts to install the **management profile** (approve it in **System Settings** → **Device Management**). Confirm the Mac appears in Intune with **Ownership = Personal**.

• **BYOD targeting (C3):** add the user/device to the BYOD Entra group (Set-ScopeTargeting) so the BYOD profile/app set applies. Keep BYOD policy lighter than corporate (respect the user's personal data — do not apply full-wipe or intrusive controls to personal devices).

• **Not compliant after enrollment:** switch to **Runbook 6** to identify the failing compliance setting.

Escalation gate

Escalate to **endpoint engineering / policy owners** when BYOD enrollment must be enabled or the BYOD policy set needs to change, or when a personal-device data-boundary question arises (what the org may and may not manage on a personal Mac). Never apply a corporate wipe to a personal device.

Evidence to collect

Enrollment restriction state (personal macOS allow/block); Company Portal installed + signed-in yes/no; **Device Management** screenshot; Intune Ownership = Personal confirmation; BYOD group membership.

Runbook 6 — Compliance & Conditional Access

Scope / symptoms

A Mac shows **non-compliant**; the user is **blocked from resources** by Conditional Access ("your device needs attention"); a Mac isn't receiving a policy/app it should. **Not covered:** the Mac isn't enrolled at all (Runbook 1/5); the account itself is locked (Runbook 7).

Triage tree

```
Is the Mac ENROLLED and visible in Intune?
■■ No  → enrollment problem → Runbook 1 (ADE) or Runbook 5 (BYOD)
■■ Yes → is it COMPLIANT?
         ■■ No       → which setting fails? → Check C2
         ■■ Yes      → getting blocked or missing a policy/app → Checks C3–C4
```

Checks

• **C1 — Presence & check-in (Intune):** Intune → **Devices** → **All devices** (OS = macOS) — note

Compliance, **Ownership**, **Last check-in**. Stale check-in → not syncing (C-sync below).

- **C2 — Compliance detail (Intune):** Intune → **Devices** → **(the Mac)** → **Device compliance** — open

the failing policy and read exactly which setting is non-compliant (common on Mac: **FileVault not on**, **minimum macOS version**, **Gatekeeper/System Integrity**, **firewall**, **password**). Cross-reference: FileVault failures → Runbook 4; OS-version failures → deploy the update.

- **C3 — Conditional Access (Entra):** Entra → **Users → (user) → Sign-in logs → Conditional

Access** tab — is a CA policy blocking because the device is non-compliant or unregistered? Unhealthy: CA "Failure" citing device compliance/state.

- **C4 — Targeting (Entra/Intune):** confirm the Mac (or its primary user) is in the Entra group

the compliance policy / config profile / app is assigned to. Missing membership = missing policy.

Checks → Resolution

- **Not syncing (C1):** Intune → device → **Sync**; on the Mac open **System Settings → Device

Management → (profile) → refresh**, or Company Portal** → Check status**. Confirm network.

- **Compliance setting (C2):** remediate the specific setting — enable FileVault (Runbook 4),

install the pending macOS update to meet minimum version, enable the firewall/Gatekeeper via the assigned profile — then re-run compliance evaluation (**Sync**).

- **CA block (C3):** once the device is compliant/registered and re-synced, the CA block clears.

If the CA policy itself must change (e.g. an exclusion is needed), that's an identity decision (gate) — do not attempt to bypass CA from the endpoint.

- **Wrong group (C4):** add the Mac/user to the correct Entra group (Set-ScopeTargeting, default

`-WhatIf`), or request the change if you lack rights.

Escalation gate

Escalate to **endpoint engineering** when a compliance setting can't be remediated on the endpoint (e.g. hardware/OS can't meet a required minimum), or the device must be retired/re-enrolled/wiped (owner approval + data backup + confirmed FileVault key). Escalate to **identity/security** when a Conditional Access policy must change.

Evidence to collect

Device Overview (Compliance / Ownership / Last check-in); the exact non-compliant setting and its policy name; sign-in log Conditional Access result; the group(s) the device is/should be in.

Runbook 7 — Incident response (encryption / SecureToken / lock-out)

Scope / symptoms

A Mac is in a **broken or locked state**: user can't unlock FileVault / forgot their password and needs the recovery key; a local account is locked out or lost its **SecureToken** (can't unlock FileVault even with the right password); the **bootstrap token** wasn't escrowed and MDM FileVault operations fail; the Mac is stuck at the FileVault login screen. **Not covered:** normal FileVault *setup/maintenance* (Runbook 4); a

compliance flag with a working Mac (Runbook 6).

> ***This is the highest-risk runbook — read the Runbook 4 safety note first.*** *Data-loss is on the table. Every branch either resolves safely or escalates *before* a destructive step.* ***Never erase or reset a Mac in this runbook without owner approval and a confirmed recovery key.***

Triage tree

```
Can the user reach the desktop at all?
■■ No (stuck at FileVault/login) → they need the recovery key or a token fix → Checks C1–C2
■■ Yes but degraded → account locked / SecureToken / bootstrap token issue   → Checks C2–C4
```

Checks

- **C1 — Recovery key available (Intune):** Intune → **Devices** → **(the Mac)** → **Recovery keys** —

is the **personal recovery key** escrowed? This is what unlocks a Mac when the password fails. Unhealthy: **no key escrowed** → escalate immediately (the Mac may not be recoverable without it).

- **C2 — Account / lock state (device, if reachable):** `dsmemberutil` / **System Settings →

Users & Groups** — is the account disabled or locked? Note repeated failed unlocks.

- **C3 — SecureToken (device):** `sysadminctl -secureTokenStatus <user>` — does the account have a

SecureToken? An account **without** SecureToken can't unlock FileVault even with the correct password. Unhealthy: SecureToken **DISABLED** for the affected user.

- **C4 — Bootstrap token (device/Intune):** `profiles status -type bootstraptoken` — is it escrowed

to MDM? Without it, MDM-driven FileVault/account operations may fail. Unhealthy: not escrowed.

Resolution

- **Locked at FileVault, password forgotten (C1):** retrieve the escrowed **personal recovery key**

from Intune → **Recovery keys**, have the user enter it at the FileVault unlock screen to boot, then reset the account password. **After** recovery, **rotate the FileVault key** (Runbook 4) so the exposed key is retired, and confirm the new key escrows. Treat the retrieved key as a secret.

- **SecureToken missing (C3):** if an admin account **with** SecureToken exists, it can grant

SecureToken to the affected user (`sysadminctl -secureTokenOn <user> ...`). **Flagged as uncertain / sensitive:** behavior varies by macOS version and Apple silicon vs. Intel — verify against Apple's documentation and do it on a pilot before production. If no admin has SecureToken, escalate (the account may need rebuild).

- **Bootstrap token not escrowed (C4):** on an ADE Apple silicon Mac, signing in as an

SecureToken-holding admin usually escrows it; then MDM FileVault operations work. Confirm with `profiles status -type bootstraptoken`. If it still won't escrow, escalate.

- **Account locked (C2):** if within your rights and the account is intact, unlock/reset per

organization policy; confirm the reset doesn't orphan FileVault (SecureToken/key state above).

- **Unrecoverable (no key, no token path):** stop. Escalate — the safe outcome may be an

owner-approved erase/re-enroll *only after* confirming what data can be recovered.

Escalation gate

Escalate to **endpoint engineering / security** immediately when: **no recovery key is escrowed** for a locked Mac; SecureToken can't be granted (no admin token-holder); the bootstrap token won't escrow; or the only remaining path is **erase/re-enroll** (owner approval + confirmed backup + confirmed key required). Any destructive step in this runbook is a gate, not a technician decision.

Evidence to collect

Recovery-key escrowed yes/no (never the key value); `sysadminctl -secureTokenStatus` result; `profiles status -type bootstraptoken` result; account lock state; exact screen/error the user sees; what was tried and the timestamp; confirmation that the key was rotated after any exposure.

Cross-runbook notes

- **Least-disruptive first, always:** Sync → local retry / re-sign-in → targeted config/group

change → reinstall → (only with approval + confirmed FileVault key + backup) erase/re-enroll. Never jump to a destructive action to save a step — on a Mac that means data loss.

- **FileVault key before any FileVault-adjacent action.** If a fix could touch encryption, the

unlock account, or the recovery key, confirm the key is escrowed **first** (Runbook 4/7 C1).

- **Scope before cause:** one user, one Mac, one location, or the whole fleet — establish this in

intake; a fleet-wide symptom almost always means policy/targeting, an expired token (ADE/VPP), or a service issue, not the single Mac in front of you.

- **The scripts support the runbooks:** use **MACRUN-Discovery** and **MACRUN-Validation** during

Checks to read Mac state at scale (enrollment, compliance, FileVault escrow, SSO/config profiles), **MACRUN-Report** to see trends for the *Document* stage, **MACRUN-Remediation** for the safe fixes (Sync / Rotate FileVault key, default `-WhatIf`), and **MACRUN-Set-ScopeTargeting** for the group-membership fixes (default `-WhatIf`).

Testing

Testing here means proving that each runbook actually leads a technician from symptom (or setup task) to a correct outcome, and that the bundled scripts read what they claim to read. Each runbook has an explicit **pass/fail gate**: the runbook "passes" when a technician following it reaches the right resolution (or the right escalation) using only the runbook's checks and evidence.

Run these on a **pilot Mac** (from the pilot Entra security group) before relying on the library across the fleet, and **re-run the enrollment, FileVault, SSO, and compliance gates on each major macOS release** (see Solution Overview — keeping current).

How to test a runbook

1. Reproduce or select a real ticket/task that matches the runbook's **Scope / symptoms**.
2. Follow the **triage tree** — confirm it lands you on the correct branch.

3. Run the **checks** in order — confirm each one is observable exactly where the runbook says.

4. Apply the **resolution / setup steps** (or reach the **escalation gate**) — confirm the outcome is correct.

1. Confirm every item in **evidence to collect** was actually capturable.

A runbook **fails** the gate if any check can't be observed as written, the triage tree misroutes, the resolution doesn't resolve/complete (or escalates when it shouldn't), or required evidence can't be captured. Fix the runbook, then re-test.

> ***FileVault safety in testing:*** *for any test that touches FileVault (Runbooks 4 and 7), confirm the pilot Mac's recovery key is escrowed **before** the test, and never use a production user's Mac for a destructive-path test.*

Pass/fail gates per runbook

Runbook 1 — ADE / enrollment

• **PASS:** A pilot serial assigned to the Intune MDM server in ABM receives its enrollment profile and enrolls through Setup Assistant; token validity and profile assignment are readable; a Mac missing its management profile is correctly routed (re-sync or re-enroll) **or** escalated with the serial, ABM assignment, and error captured.

• **FAIL:** Enrollment failure treated without checking ABM assignment/token; a Mac is erased to "fix" enrollment without approval and a confirmed recovery key.

Runbook 2 — SSO setup

• **PASS:** The Enterprise SSO plug-in (or Platform SSO) profile is confirmed assigned and installed; Company Portal presence verified; a registration/token failure is distinguished from a CA block via the sign-in log; SSO works after setup **or** a CA/Platform-SSO-design issue is escalated.

• **FAIL:** No check of profile assignment or Company Portal; MFA/CA change attempted on the endpoint; Platform SSO rolled out fleet-wide without a pilot.

Runbook 3 — App deployment (VPP / LOB)

• **PASS:** VPP token validity and license count checked (for VPP); assignment/targeting and install status read; deploy-vs-runtime correctly distinguished; a targeting miss fixed via group membership (not sideloading); a failed install resolved by re-sync/detection fix **or** escalated with the error captured.

• **FAIL:** App sideloaded to bypass a broken deployment; no install error or token/license state captured.

Runbook 4 — Passwordless / FileVault

• **PASS:** FileVault policy assignment and `fdesetup status` confirmed; **recovery key escrow verified before any change**; a missing key is escrowed via rotate; SecureToken/bootstrap-token state read and reconciled; passwordless/Platform SSO stood up on the pilot.

• **FAIL:** Any FileVault change attempted without a confirmed escrowed key; recovery key value written into a ticket; SecureToken granted in production without pilot verification.

Runbook 5 — BYOD

• **PASS:** Enrollment restriction (personal macOS allow) and Company Portal presence checked; the Mac enrolls user-driven and shows **Ownership = Personal**; BYOD group targeting verified; a non-compliant BYOD Mac is handed to Runbook 6.

• **FAIL:** A corporate/full-wipe control applied to a personal Mac; enrollment attempted without confirming BYOD is allowed and the user is licensed.

Runbook 6 — Compliance & Conditional Access

• **PASS:** Presence/check-in confirmed; the exact non-compliant setting is identified by name (e.g. FileVault, minimum macOS); CA block distinguished from plain non-compliance via the sign-in log; setting remediated and re-evaluated **or** escalated; targeting verified when a policy is missing.

• **FAIL:** "Non-compliant" reported without naming the failing setting; a CA bypass attempted on the endpoint; targeting not checked when a policy is missing.

Runbook 7 — Incident response

• **PASS:** Recovery-key escrow is the first check; a locked Mac is recovered with the escrowed key and the key is **rotated afterward**; SecureToken/bootstrap-token state is read and either fixed on a pilot or escalated; **no erase/reset without approval + confirmed key**.

• **FAIL:** A locked Mac with no escrowed key is worked toward an erase without escalation; the recovery key is captured into a ticket; a destructive step is taken as a technician decision.

Script tests (read-only, run first)

• **MACRUN-Discovery.ps1** — PASS when it connects to Graph with read scopes and returns Mac health/context (enrollment, compliance, FileVault-escrow indicator, OS, primary user) for the target device(s) without writing anything. FAIL if it errors on scopes or attempts a write.

• **MACRUN-Validation.ps1** — PASS when the per-device state it reports (enrollment, compliance with named non-compliant policies, assigned config profiles, recovery-key escrow) matches what the Intune portal shows for a pilot Mac.

• **MACRUN-Report.ps1** — PASS when the support-metrics / fleet-health summary runs read-only and its figures reconcile with the portal for the pilot scope.

• **MACRUN-Remediation.ps1** — PASS when run with the default `-WhatIf`: it prints the intended safe action (Sync / Rotate FileVault key) and makes **no** change. Only after WhatIf review should it be run for real on a pilot Mac.

• **MACRUN-Set-ScopeTargeting.ps1** — PASS when run with the default `-WhatIf`: it prints the intended

group membership change and makes **no** change until explicitly confirmed on the pilot group.

Overall gate to leave pilot: every runbook passes on the pilot Mac, all five scripts pass their read-only/`-WhatIf` tests, the FileVault recovery key is confirmed escrowed for the pilot Mac, and the evidence lists are all capturable in your ticketing tool.

Troubleshooting

The Runbook Library covers the common single-category cases. This guide handles the **cross-cutting edge cases** — tasks that span two runbooks, symptoms that mimic one cause but have another, and the failure modes of the tooling and the macOS platform itself. Use it when a ticket doesn't sit cleanly inside one runbook.

Symptoms that cross runbook boundaries

• **"Can't sign in to apps on my Mac" that is really SSO.** Microsoft 365 apps re-prompt for

password/MFA. This is **Runbook 2 (SSO)**, not an app problem — check the SSO extension/Platform SSO payload and the sign-in log before touching the app (Runbook 3).

• **"Blocked from resources" — Conditional Access vs. compliance.** Being blocked can come from a

non-compliant Mac (Runbook 6) *or* a Conditional Access policy (Runbook 1/2 identity paths). The sign-in log's Conditional Access tab names which policy applied; that decides the runbook.

• **"Non-compliant Mac" that is really FileVault or an OS version.** Compliance fails on "FileVault

not enabled" or "minimum macOS version." Start in **Runbook 6** to name the failing setting, then switch to **Runbook 4** (FileVault) or deploy the macOS update, then re-evaluate compliance.

• **"FileVault won't turn on" that is really a missing bootstrap/SecureToken.** The FileVault policy

is assigned but encryption never engages or the key never escrows. Check **SecureToken and the bootstrap token** (Runbook 4 C4 / Runbook 7 C3–C4) — MDM-driven FileVault needs them.

• **"App installed but broken" that is really licensing.** A VPP app shows installed but won't

activate — check **VPP licenses and token** (Runbook 3 C1–C2) before assuming a runtime fault.

Confusing single-symptom cases

• **Mac stuck at the FileVault unlock screen.** Do **not** improvise. Go straight to **Runbook 7 C1**

— is the recovery key escrowed? If yes, recover with it and rotate afterward. If no, escalate; this is potentially unrecoverable and is never a guess-the-password exercise.

• **User can log in but can't unlock FileVault.** Classic **SecureToken missing** (Runbook 7 C3):

the password is correct but the account has no SecureToken, so it can't unlock the encrypted disk.

• **"My Mac isn't managed."** Confirm with `profiles status -type enrollment` and **System Settings →

Device Management **before concluding it's unenrolled — the MDM profile may be present but the Mac simply hasn't checked in (trigger** Sync**), or you may be searching the wrong identifier (serial vs. name vs. primary user — Requirement R4).

• **New Mac never offered enrollment in Setup Assistant.** ADE only triggers during Setup Assistant.

If the Mac was already past setup, it must be **erased/reset** to re-run it (Runbook 1 gate — approval + backup + confirmed key) or enrolled user-driven via Company Portal (Runbook 5).

• **Repeated MFA prompts on a Mac.** Often clock skew (token validation fails when the Mac's clock is

wrong — check date/time first), then a broken SSO registration (Runbook 2 C3), then a Conditional Access session policy.

When scope changes the answer

• **One Mac vs. many.** If several Macs show the same symptom at once, stop working the single device:

it's almost certainly **policy/targeting** (wrong or missing Entra group — use MACRUN-Set-ScopeTargeting to inspect) or an **expired token** (ADE or VPP — Runbook 1/3). Working each Mac individually will not fix a fleet-wide cause.

• **New enrollment vs. established.** A brand-new ADE enrollment failing differently from established

Macs usually points to the ABM assignment, the ADE token, or the enrollment profile (Runbook 1), not the everyday support paths.

• **After a macOS major upgrade.** A symptom that appears right after a macOS release often means a

deprecated MDM command or a changed Platform SSO/FileVault behavior — re-verify against the current Apple/Intune docs and re-run the Testing gates before assuming the Mac is broken.

Tooling and script edge cases

• `Connect-MgGraph` **fails or lacks scopes.** Re-run with the explicit `-Scopes` the script header

lists; if consent is required, an admin must grant it once (Requirement R2). `Get-MgContext` confirms the active scopes.

• **A script returns fewer Macs than expected.** The signed-in account's Intune **scope (groups)**

may limit what it can read (Requirement R1). Confirm the technician's role scope includes the target Macs. Also confirm the filter — the scripts filter to `operatingSystem eq 'macOS'`.

• **Remediation/Set-ScopeTargeting "did nothing."** By design — both default to `-WhatIf` and only

print the planned change. Re-run without `-WhatIf` (on a pilot first) once you've reviewed the plan.

• **Rotate FileVault key "did nothing."** With the default `-WhatIf` the Remediation script only

previews. When run for real, allow the Mac to check in, then re-run MACRUN-Validation and confirm the **new** key is escrowed in Intune before considering the old one retired.

• **Portal and script disagree.** The portal can lag; trigger **Sync** on the Mac and re-run the

read-only Validation script. Persistent disagreement is itself evidence — attach both and escalate.

- **Remote tool won't connect / can't control the Mac.** Confirm the approved remote tool is deployed and its macOS **Screen Recording / Accessibility (PPPC)** permissions are granted (Requirement R3), the user is present to consent, and the Mac has connectivity before assuming a tool fault.

When nothing fits

If a ticket matches no runbook and no edge case above: capture the standard evidence (device name/serial, exact symptom, scope, timestamps, any error codes, relevant `profiles`/`fdesetup` output), run MACRUN-Discovery/Validation for state, and escalate to endpoint engineering with that package. An unclassifiable-but-recurring ticket is a signal to add a new runbook — feed it back into the library (see Best Practices).

Best Practices

These practices keep the runbook library working as intended and turn it into a genuine **enablement kit**: **documentation** that stays current, **training** that spreads the knowledge off one engineer's head, and **feature evaluation** that adopts new Intune/Apple capabilities deliberately. They apply across every runbook.

Documentation

- **Capture as you go, not at the end.** Screenshot the failing signal at the moment you see it — compliance setting, sign-in log reason, `fdesetup status`, `profiles status` output, error code.
- **Use the runbook's evidence list as a checklist.** Every runbook ends with an explicit *evidence to collect* list; treat it as the definition of "documented."
- **Record IDs, serials, command output, and timestamps** — not just descriptions. The Intune device name/serial, the exact non-compliant setting, `0x…`/exit codes, and precise times let the next person (or an escalation target) continue without redoing your diagnosis.
- **Prefer read-only tooling for capture.** Use MACRUN-Discovery/Validation/Report and the Intune portal to gather state; they don't change anything and are safe to run repeatedly.
- **Never capture secrets.** Do **not** screenshot or paste **FileVault recovery keys**, passwords, bootstrap-token material, or full VPP/ADE tokens into a ticket. Reference *that* a key is escrowed, never its value.
- **Keep the runbooks themselves current.** When a menu moves, a command is deprecated, or Apple/Intune changes a behavior, edit the affected runbook the same day. macOS changes faster than Windows — a stale runbook is worse than none because it sends technicians down a wrong path.

Training and enablement

- **Onboard new technicians on the operating model first.** Teach intake → triage → diagnose → resolve/escalate → document once; it applies to every runbook, so a new hire is productive after learning the shape, not all seven runbooks at once.
- **Pair the first runs.** Have a new technician shadow, then run a live-but-supervised enrollment, an SSO setup, and a compliance fix on a pilot Mac before working real Mac tickets solo.
- **Rehearse the incident-response runbook off-hours.** Runbook 7 (FileVault lock-out / SecureToken) is the one you cannot afford to improvise under pressure. Walk it on a test Mac — recover with an escrowed key, rotate it, and confirm — so the steps are muscle memory before a real lock-out.
- **Build a short internal knowledge base from the evidence.** Real tickets, redacted of secrets, become the best training material — they show the checks producing real signals.
- **Cross-train beyond one engineer.** The whole point of the library is that Mac knowledge stops living in one person's head. Rotate who runs enrollment, SSO, and IR so no single person is the bus factor.

Feature evaluation (adopting new capabilities deliberately)

- **Track the release notes as a standing task.** Watch Intune "What's new" and Apple's macOS/ABM release notes. New capabilities (a new Platform SSO mode, a new declarative-management setting, a new compliance control) are opportunities — but only after evaluation.
- **Evaluate before you enable.** For any new feature, ask: what problem does it solve for us, what does it change for the user, what are the prerequisites (macOS version, hardware, licensing), and what is the rollback? Write that down before touching production.
- **Pilot every feature on the pilot group first.** Enable it for the pilot Entra group, run the relevant Testing gate, and confirm it behaves as documented on real hardware before fleet rollout — Platform SSO and FileVault changes especially.
- **Flag uncertainty honestly.** Where a feature's behavior is version-dependent or the documentation is thin, say so in the runbook and verify against the vendor docs rather than guessing. (This library flags several such spots — Platform SSO options, SecureToken grants.)
- **Add a runbook when a new feature becomes routine.** Once a new capability is adopted and supported repeatedly, give it its own runbook (or a branch of an existing one) so support stays consistent.

Escalation

- **Escalate at the gate, not before or after.** Each runbook names the exact conditions that mean *stop and hand off*. Escalating earlier drops ownership; escalating later delays the user.
- **Escalate with a complete package.** Hand over the collected evidence, what you already tried, and the specific decision or right the target needs to supply. A good escalation is one the receiver can act on without a follow-up question.

• **Route to the right team.** Identity/security for accounts, MFA, Conditional Access, and Platform SSO design; endpoint engineering for ADE tokens, enrollment, FileVault/SecureToken recovery, and re-enroll; app owner/vendor for packaging or product-internal faults; the ABM owner for ABM/VPP token and Apple Configurator work.

• **Own the ticket across the hand-off.** Escalation is a transfer of *action*, not of *responsibility for the user's experience* — track it to resolution.

Live-environment safety (Mac specifics)

• **FileVault key before anything FileVault-adjacent.** Confirm the recovery key is escrowed **before** any action that could touch encryption, the unlock account, or the key. No exceptions.

• **Default to `-WhatIf`.** The Remediation and Set-ScopeTargeting scripts ship WhatIf-first; review the planned change before removing it, and run any state change on a **pilot** Mac or group first.

• **Confirm data backup before anything destructive.** Erase/reset, re-enrollment, and account rebuilds require owner approval **and** confirmed backup **and** a confirmed escrowed recovery key.

• **Change targeting, not the endpoint, to fix "missing policy/app."** If a Mac isn't getting a profile or app, fix its Entra group membership rather than sideloading or forcing settings locally.

Keeping the library healthy

• **Feed corrections back.** When a runbook step is wrong (a menu moved, a command deprecated, a check no longer shows what's described), update the runbook so the next technician benefits.

• **Watch the trend.** Use MACRUN-Report over time — a rising category is a signal to add or refine a runbook (or to evaluate a feature that would prevent the tickets), not just to work more tickets.

Deliverables Summary

This solution is a self-contained **runbook library and IT-support enablement kit** plus supporting scripts for managing and supporting Intune-managed Macs. Everything below is yours to keep and reuse across the fleet.

Documents

• **Solution Overview** (`01-solution-overview.md`) — the macOS support operating model (intake → triage → diagnose → resolve/escalate → document), how it maps to an Intune-managed Mac fleet (Intune, Entra, Apple Business Manager, the Mac), the fixed anatomy every runbook uses, and how to keep the library current with Intune/Apple changes.

• **Requirements Guide** (`02-requirements.md`) — the technician prerequisites (Intune read role, Graph read access, a macOS remote-control tool, Mac asset/user context, a place to record outcomes), each with Description / Why / Verify / Implement-if-missing, plus the category-level global requirements

read with the macOS note.

- **Runbook Library** (`03-implementation.md`) — the seven categorized runbooks — ADE/enrollment,

SSO setup, app deployment (VPP/LOB), passwordless/FileVault procedures, BYOD, compliance & Conditional Access, and incident response (encryption/SecureToken/lock-out) — each with triage tree, checks, resolution/steps, escalation gate, and evidence to collect.

- **Testing Guide** (`05-testing.md`) — pass/fail gates per runbook and per script, run on a pilot

Mac before fleet use and re-run on each major macOS release.

- **Troubleshooting Guide** (`06-troubleshooting.md`) — cross-cutting edge cases that span runbooks

and macOS-specific pitfalls.

- **Best Practices** (`07-best-practices.md`) — documentation, training/enablement, and deliberate

feature evaluation, plus Mac-specific live-environment safety.

- **Global Requirements — Infrastructure** (`global-requirements-infrastructure.md`) — the

category-level prerequisites shared by all Infrastructure solutions.

PowerShell scripts

All scripts read from Microsoft Graph with read-only scopes; the two that can change state default to `-WhatIf`. Every script filters to macOS devices.

- **MACRUN-Discovery.ps1** — Mac fleet health/context discovery across target devices

(enrollment, compliance, FileVault-escrow indicator, OS, primary user) (read-only).

- **MACRUN-Validation.ps1** — per-device state validation (enrollment, compliance with named

non-compliant policies, assigned config profiles, recovery-key escrow) (read-only).

- **MACRUN-Report.ps1** — support-metrics / Mac-fleet-health summary report (read-only).
- **MACRUN-Remediation.ps1** — safe common fixes (Sync, Rotate FileVault key); defaults to `-WhatIf`.
- **MACRUN-Set-ScopeTargeting.ps1** — build/adjust pilot and support target groups; defaults to

`-WhatIf`.

How the pieces fit together

Read the **Overview** to learn the operating model, confirm the **Requirements**, then work Mac tasks and tickets from the **Runbook Library**. Use **Discovery/Validation/Report** during the *Checks* and *Document* stages, **Remediation** for the safe fixes (including Rotate FileVault key), and **Set-ScopeTargeting** to correct group-based targeting. Prove everything on a pilot Mac using the **Testing** gates, lean on **Best Practices** for documentation/training/feature evaluation, and reach for **Troubleshooting** when a ticket doesn't fit a single runbook.

Outcome

A Mac-support team that resolves issues the same repeatable way and adopts new features deliberately — a runbook library, an enablement kit, and scripts you keep and reuse across the fleet, at a fraction of the cost of ad-hoc consulting.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

MACRUN-Discovery.ps1

Purpose: Discovers health and support context across the Intune-managed Mac fleet so a technician can triage at scale instead of clicking each device. Pulls managed-device inventory, ownership, compliance state, last check-in, macOS build, and primary user from Microsoft Graph, and flags whether a FileVault personal recovery key is escrowed. Read-only: it never changes the tenant. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MACRUN-Discovery.ps1

How to run: powershell -ExecutionPolicy Bypass -File .\MACRUN-Discovery.ps1

MACRUN-Remediation.ps1

Purpose: Applies the safe, least-disruptive common fixes the runbooks call for on a Mac: trigger an Intune device Sync (re-check-in / re-evaluate compliance), or Rotate the FileVault personal recovery key (generate and escrow a fresh key). It reads device state from Microsoft Graph, then issues only the low-risk Intune device action requested. It defaults to -WhatIf and makes no change until you re-run with -WhatIf:$false. It never wipes, erases, resets, retires, or deletes anything, and it never disables FileVault. HOW TO RUN Preview (no change, default): powershell -ExecutionPolicy Bypass -File .\MACRUN-Remediation.ps1 -DeviceName device-name-or-serial -Action Sync Apply after reviewing: powershell -ExecutionPolicy Bypass -File .\MACRUN-Remediation.ps1 -DeviceName device-name-or-serial -Action Sync -WhatIf:$false

How to run: Preview (no change, default): powershell -ExecutionPolicy Bypass -File .\MACRUN-Remediation.ps1 -DeviceName device-name-or-serial -Action Sync Apply after reviewing: powershell -ExecutionPolicy Bypass -File .\MACRUN-Remediation.ps1 -DeviceName device-name-or-serial -Action Sync -WhatIf:$false

MACRUN-Report.ps1

Purpose: Produces a support-metrics and Mac-fleet-health summary a lead can use to spot trends: counts by compliance state, stale (not-checked-in) Macs, macOS-version spread, and ownership mix (corporate vs. personal/BYOD), drawn from Microsoft Graph. Read-only: it never changes the tenant. Use it during the Document stage and to decide whether a rising category needs a new or refined runbook, or a feature evaluation. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MACRUN-Report.ps1

How to run: powershell -ExecutionPolicy Bypass -File .\MACRUN-Report.ps1

MACRUN-Set-ScopeTargeting.ps1

Purpose: Fixes the group-based targeting the runbooks rely on: ensures a pilot/support Entra security group exists and adds the specified Mac (its Entra device object) to it, so the Mac starts receiving the configuration profile or app it was missing. It reads current membership from Microsoft Graph and only adds the member you name. It defaults to -WhatIf and makes no change until you re-run with -WhatIf:$false. It never removes members or deletes groups. HOW TO RUN Preview (no change, default): powershell -ExecutionPolicy Bypass -File .\MACRUN-Set-ScopeTargeting.ps1 -GroupName pilot-group-name -DeviceName device-name-or-serial Apply after reviewing: powershell -ExecutionPolicy

Bypass -File .\MACRUN-Set-ScopeTargeting.ps1 -GroupName pilot-group-name -DeviceName device-name-or-serial -WhatIf:$false

How to run: Preview (no change, default): powershell -ExecutionPolicy Bypass -File .\MACRUN-Set-ScopeTargeting.ps1 -GroupName pilot-group-name -DeviceName device-name-or-serial Apply after reviewing: powershell -ExecutionPolicy Bypass -File .\MACRUN-Set-ScopeTargeting.ps1 -GroupName pilot-group-name -DeviceName device-name-or-serial -WhatIf:$false

MACRUN-Validation.ps1

Purpose: Validates the support-relevant state of a single Mac so a technician can confirm what a runbook check expects. Reports enrollment/ownership state, compliance (with the specific non-compliant policies named), assigned configuration profiles, and whether a FileVault personal recovery key is escrowed, from Microsoft Graph. Read-only: it never changes the tenant. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MACRUN-Validation.ps1 -DeviceName device-name-or-serial

How to run: powershell -ExecutionPolicy Bypass -File .\MACRUN-Validation.ps1 -DeviceName device-name-or-serial